

HIGH PERFORMANCE COMPUTING

UNIT 6

High Performance Computing
Applications



SCOPE OF PARALLEL COMPUTING

- Parallel computing has a wide scope in modern technology because it allows multiple tasks to run simultaneously, increasing speed and efficiency. It is used in areas where huge amounts of data and complex calculations are involved.
- With the growth of multi-core processors, GPUs, cloud computing, and artificial intelligence, parallel computing has become an essential part of modern computing systems.

Major Areas of Application

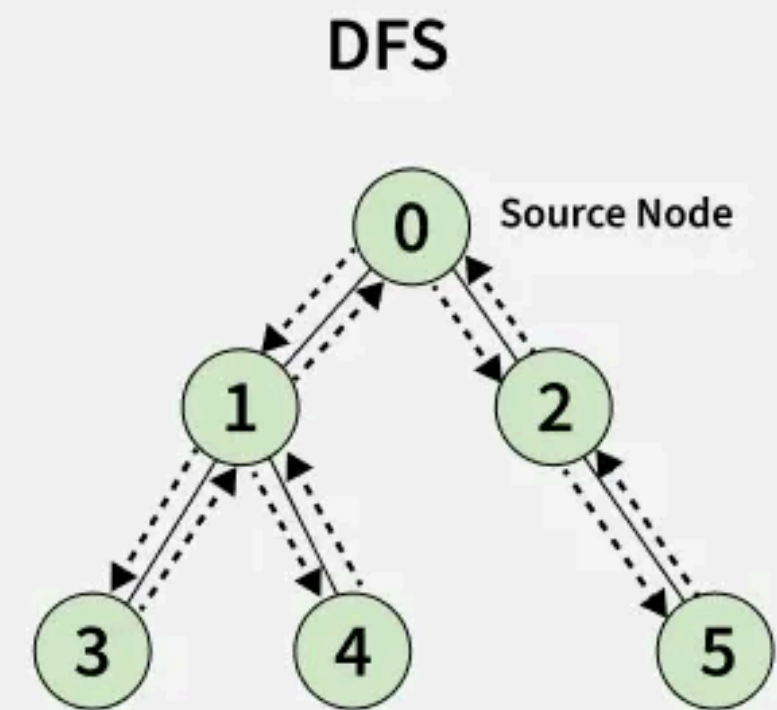
- **Scientific Research** – Used in weather forecasting, space research, simulations, and molecular modeling.
- **Artificial Intelligence & Machine Learning** – Helps in training deep learning models faster.
- **Big Data Analytics** – Processes large datasets efficiently in less time.
- **Graphics & Gaming** – Improves rendering, animation, and real-time gaming performance.
- **Cloud Computing** – Enables multiple users and services to work simultaneously.
- **Medical Field** – Used in MRI scans, genome analysis, and drug discovery.
- **Cyber Security** – Helps in encryption, threat detection, and network monitoring.
- **Business Applications** – Supports online banking, e-commerce, and large-scale transactions.

PARALLEL SEARCH ALGORITHMS: DEPTH FIRST SEARCH (DFS)

Depth First Search (DFS) is a graph traversal algorithm used to explore nodes deeply before moving to another branch. In Parallel DFS, multiple processors or threads work together to explore different parts of the search tree simultaneously, which reduces execution time and improves performance.

Working of Parallel DFS

- Start from the root node.
- Divide child nodes among different processors.
- Each processor explores its assigned branch deeply.
- If a processor finishes early, it may take work from another processor (load balancing).
- The search continues until the target node is found or all nodes are visited.



Output: 0, 1, 3, 4, 2, 5

PARALLEL SEARCH ALGORITHMS: DEPTH FIRST SEARCH (DFS)

```
ParallelDFS(node)
{
    Mark node as visited

    Parallel for each adjacent node
    {
        If node is not visited
            ParallelDFS(adjacent node)
    }
}
```

Advantages

- Faster search in large graphs and trees.
- Better utilization of multi-core processors.
- Reduces overall execution time.
- Useful for large-scale AI and scientific applications.

Disadvantages

- Difficult synchronization between processors.
- Load balancing can be challenging.
- Requires extra memory and communication overhead.

PARALLEL SEARCH ALGORITHMS: BREADTH FIRST SEARCH (BFS)

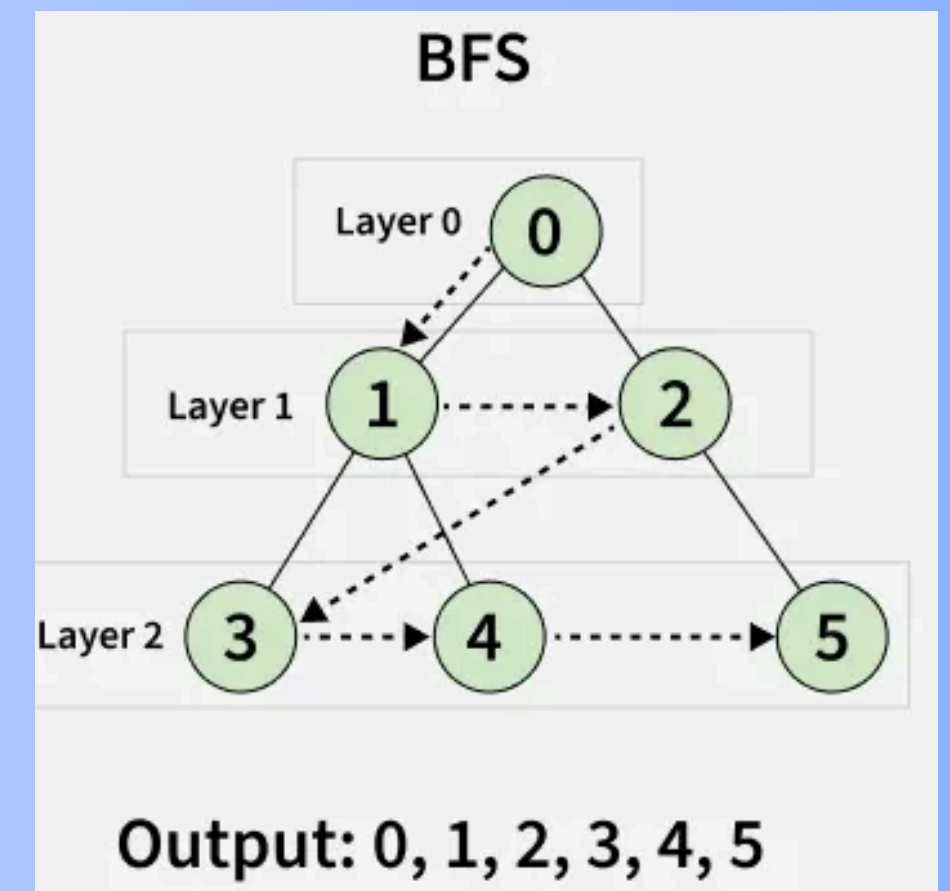
Breadth First Search (BFS) is a graph traversal algorithm that explores all neighboring nodes level by level before moving to the next depth level. In Parallel BFS, multiple processors work together to traverse different nodes simultaneously, improving performance for large graphs.

BFS for Parallel Execution

In Parallel BFS, graph nodes are divided among processors. Each processor explores nodes at the current level in parallel and then moves to the next level together.

Working Steps

1. Start from the source node.
2. Add the source node to the queue.
3. Multiple processors simultaneously process nodes of the same level.
4. Each processor visits adjacent unvisited nodes.
5. Newly discovered nodes are added to the next-level queue.
6. Continue until all nodes are visited.



PARALLEL SEARCH ALGORITHMS: BREADTH FIRST SEARCH (BFS)

```
Parallel BFS(Graph G, Source S)
{
    Mark S as visited
    Add S to queue

    while(queue is not empty)
    {
        Parallel for each node in current level
        {
            Visit adjacent nodes

            if(adjacent node not visited)
            {
                Mark visited
                Add to next queue
            }
        }

        Move to next level
    }
}
```

Advantages of Parallel BFS

- Faster traversal for large graphs.
- Efficient for social networks and web graphs.
- Better CPU utilization.
- Reduces execution time.

Disadvantages

- High communication overhead.
- Synchronization complexity.
- Uneven workload may occur.

COMMUNICATION STRATEGIES FOR PARALLEL BFS

1. Message Passing

- Processors send discovered nodes to other processors.
- Commonly implemented using MPI.
- Suitable for distributed memory systems.

Example

Processor P1 discovers a node owned by P2, so P1 sends that node information to P2.

2. Shared Memory Communication

- All processors access shared data structures like queues and visited arrays.
- Synchronization mechanisms (locks/semaphores) are required.

Used In

Multi-core systems using OpenMP or threads.

3. Level Synchronization

- All processors complete one BFS level before moving to the next.
- Ensures correct traversal order.

4. Load Balancing

- Work is distributed evenly among processors.
- Idle processors may take nodes from busy processors.

COMPLEXITY ANALYSIS OF PARALLEL BFS

Let:

- V = Number of vertices
- E = Number of edges
- P = Number of processors

Sequential BFS Complexity

- $O(V+E)$

Parallel BFS Complexity

- Ideally, the work is divided among processors:
- $O\left(\frac{V+E}{P}\right)$

Additional Overhead

- Communication and synchronization overhead may increase execution time.

Space Complexity

- $O(V)$

for storing visited nodes and queues.

PARALLEL SORTING: BUBBLE SORT

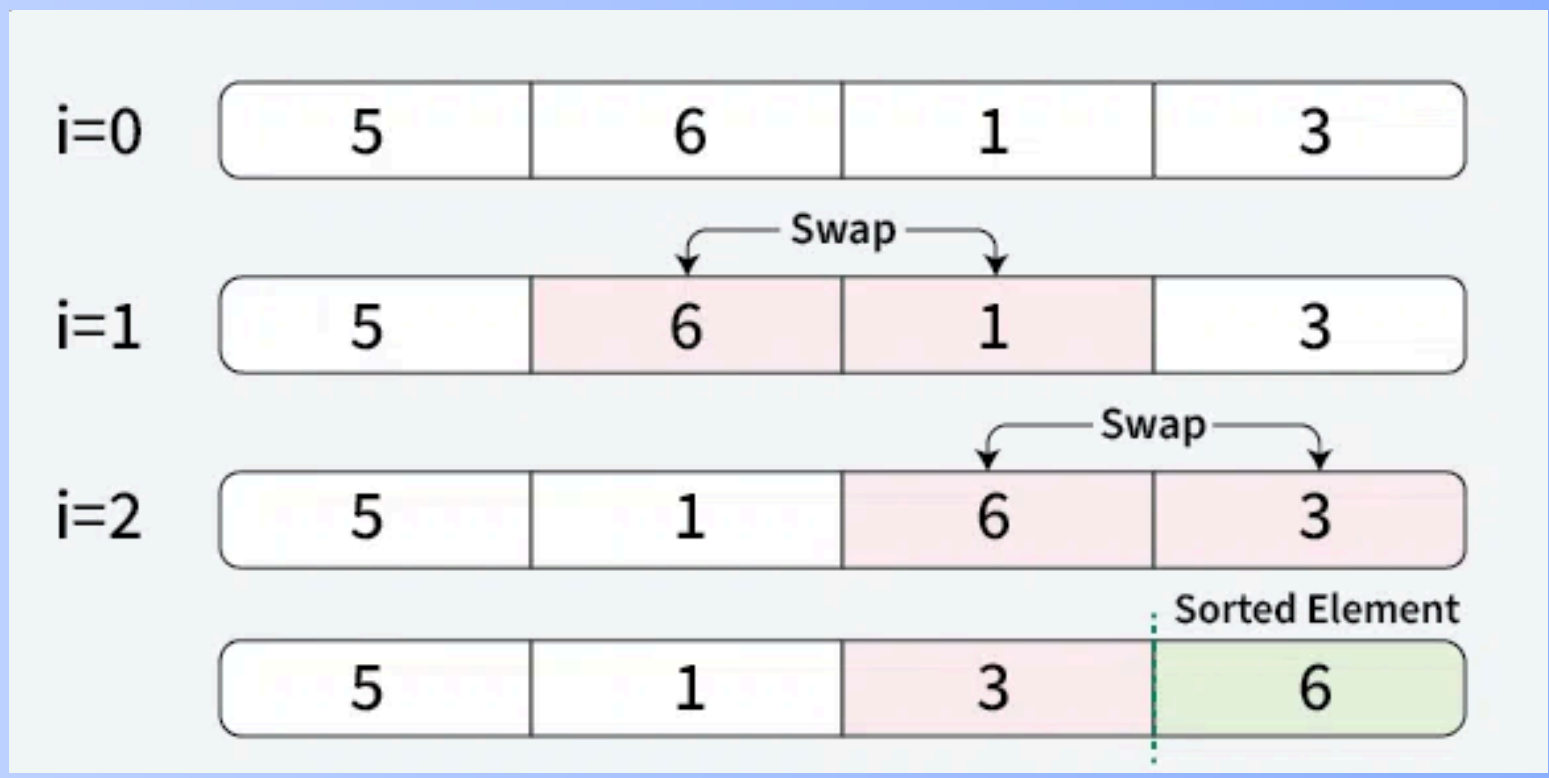
Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. In Parallel Bubble Sort, different pairs of elements are processed at the same time.

Working of Parallel Bubble Sort

The algorithm works in phases:

- 1. Compare adjacent elements simultaneously.
- 2. Swap elements if they are in the wrong order.
- 3. Repeat the process for all phases until the array becomes sorted.

Usually, the Odd-Even Transposition Sort method is used for parallel execution.



PARALLEL SORTING: BUBBLE SORT

Odd-Even Parallel Bubble Sort

Odd Phase

Compare:

- A[1] with A[2]
- A[3] with A[4]
- A[5] with A[6]

These comparisons happen simultaneously.

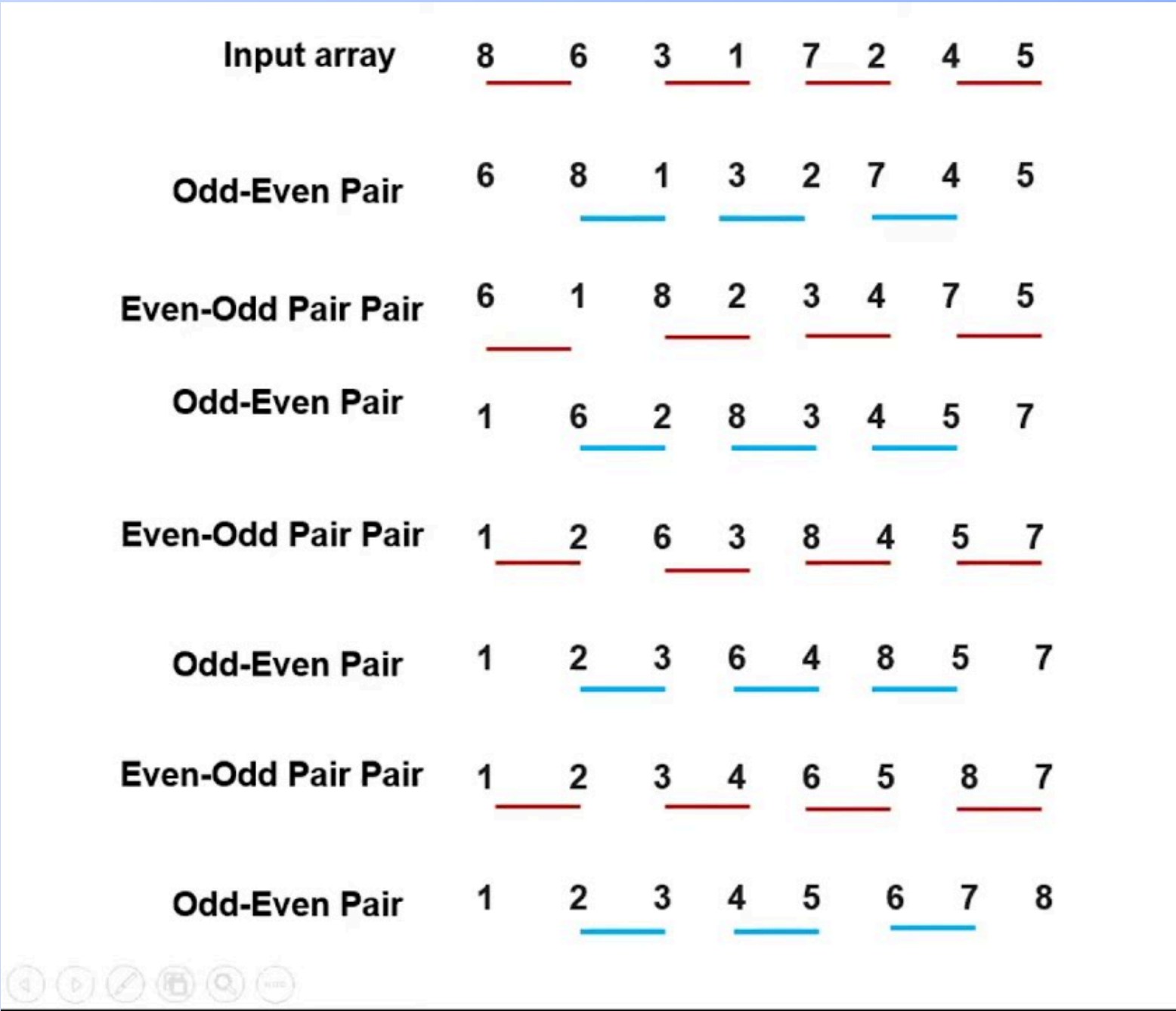
Even Phase

Compare:

- A[0] with A[1]
- A[2] with A[3]
- A[4] with A[5]

Again, all comparisons occur in parallel.

This process repeats until the array is sorted.



PARALLEL SORTING: BUBBLE SORT

```
Parallel Bubble Sort(A, n)
{
    for phase = 0 to n-1
    {
        if phase is even
        {
            Parallel compare and swap
            (A[0],A[1]), (A[2],A[3]) ...
        }
        else
        {
            Parallel compare and swap
            (A[1],A[2]), (A[3],A[4]) ...
        }
    }
}
```

Complexity Analysis

Sequential Bubble Sort : $O(n^2)$

Parallel Bubble Sort

With multiple processors performing comparisons simultaneously:

- $O(n)$

under ideal parallel conditions.

Space Complexity : $O(1)$

PARALLEL SORTING: MERGE SORT

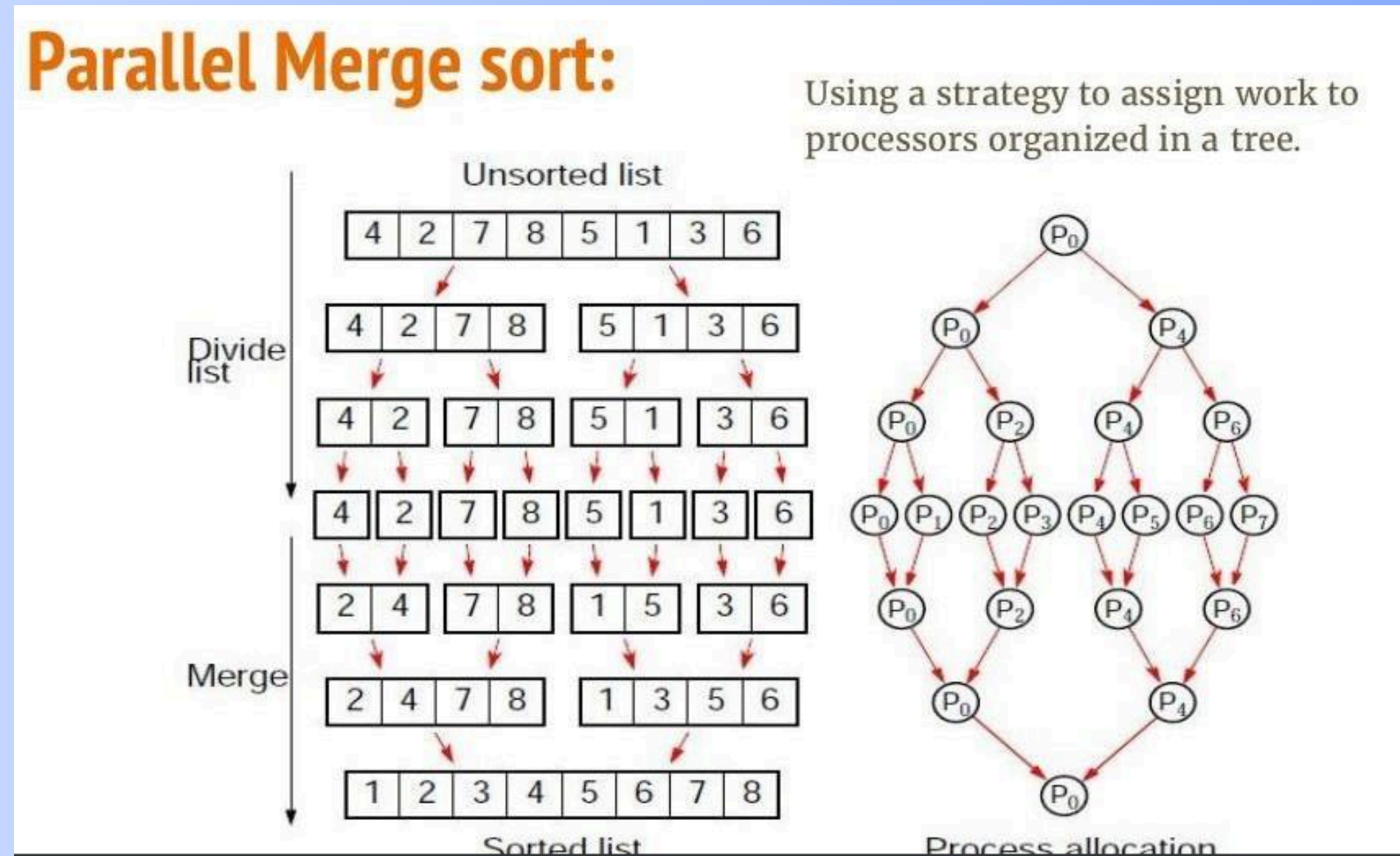
Parallel Merge Sort is a parallel version of the Merge Sort algorithm where multiple processors divide and sort different parts of the array simultaneously. It follows the Divide and Conquer approach.

The array is divided into smaller subarrays, each processor sorts its assigned part independently, and then the sorted subarrays are merged together to form the final sorted array.

Working of Parallel Merge Sort

Steps

1. Divide the array into smaller subarrays.
2. Assign subarrays to different processors.
3. Each processor sorts its part recursively.
4. Merge sorted subarrays in parallel.
5. Continue merging until the complete array becomes sorted.



Process allocation

PARALLEL SORTING: MERGE SORT

```
ParallelMergeSort(A)
{
    if(size of A <= 1)
        return

    Divide A into two halves

    Parallel execute
    {
        ParallelMergeSort(left half)
        ParallelMergeSort(right half)
    }

    Merge both sorted halves
}
```

Complexity Analysis

Sequential Merge Sort

- $O(n \log n)$

Parallel Merge Sort

- With P processors:

$$O\left(\frac{n \log n}{P}\right)$$

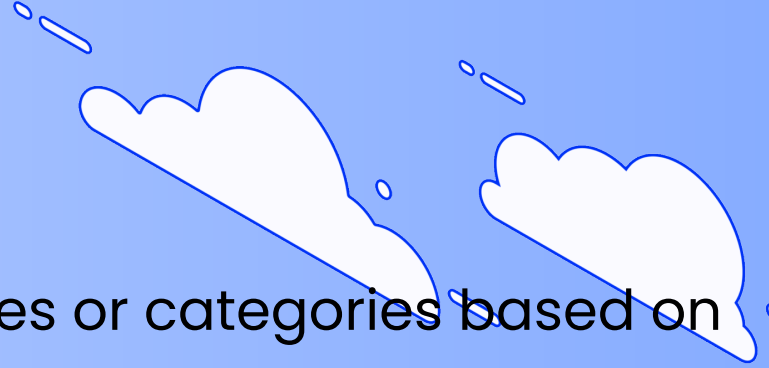
- under ideal conditions.

Space Complexity

- $O(n)$

because extra memory is needed during merging.

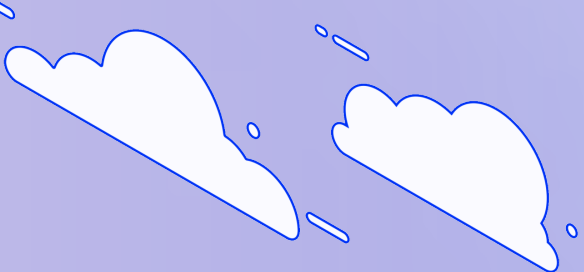
DISTRIBUTED COMPUTING: DOCUMENT CLASSIFICATION



- Document Classification is the process of automatically categorizing documents into predefined classes or categories based on their content. In Distributed Computing, this task is divided among multiple computers or processors to improve speed, scalability, and efficiency when handling large amounts of text data.
- Distributed document classification is widely used in search engines, spam filtering, sentiment analysis, and digital libraries.

Working of Distributed Document Classification

Steps

1. Collect large numbers of documents.
 2. Divide documents among multiple nodes/processors.
 3. Each processor extracts features such as keywords, frequency, or patterns.
 4. Machine learning algorithms classify documents into categories.
 5. Results from all processors are combined to produce the final output.
- 

FRAMEWORKS - KUBERNETES

- Kubernetes is an open-source container orchestration framework used to automate the deployment, scaling, and management of containerized applications. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation.
- Kubernetes helps manage distributed applications across multiple machines or nodes efficiently. It is widely used in cloud computing, DevOps, and distributed systems.

Features of Kubernetes

1. Container Orchestration

- Automatically manages containers across clusters.

2. Auto Scaling

- Increases or decreases resources based on workload.

3. Load Balancing

- Distributes traffic evenly among containers.

4. Self-Healing

- Automatically restarts failed containers.

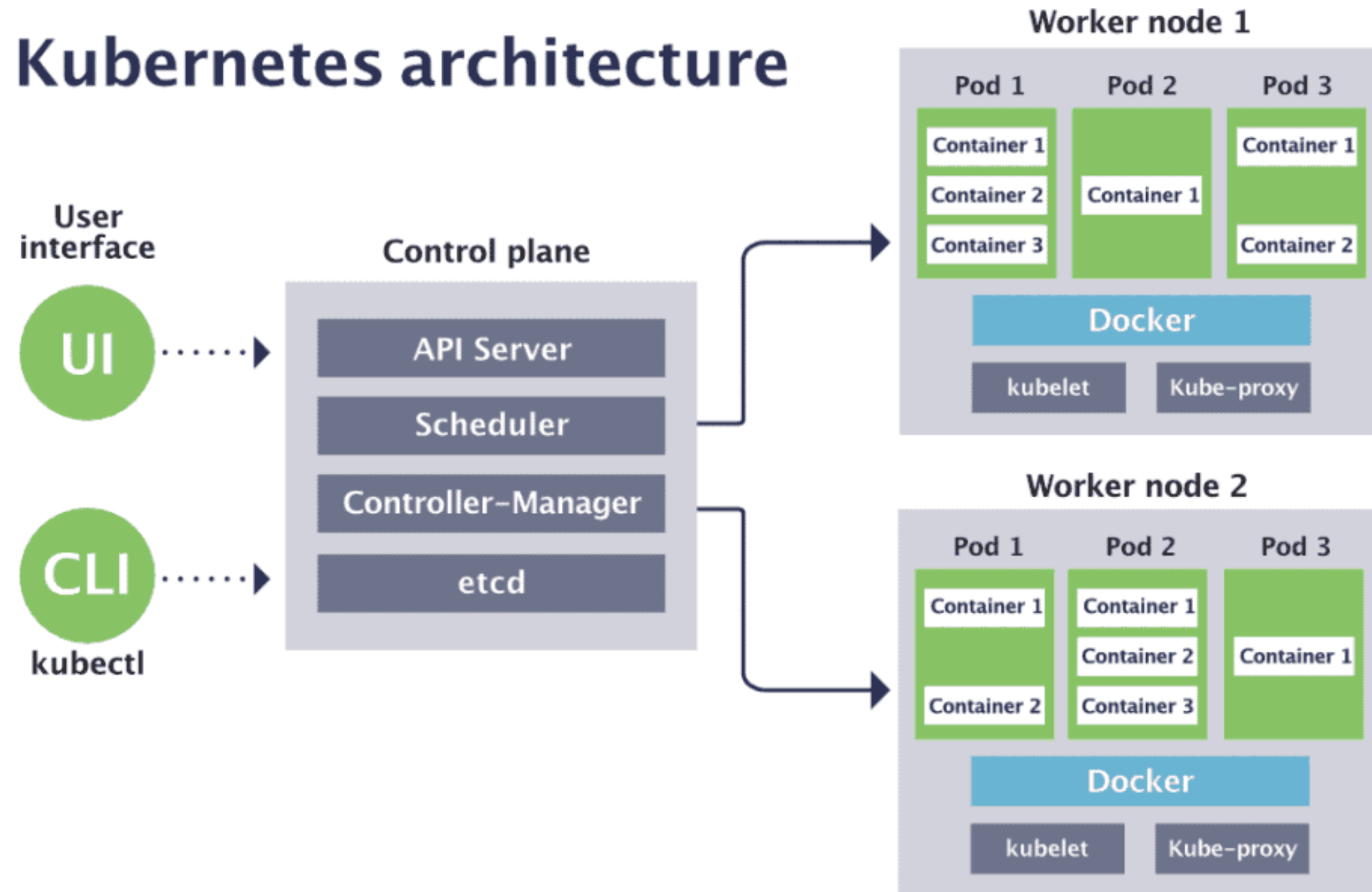
5. Service Discovery

- Helps applications communicate easily inside the cluster.

6. Rolling Updates

- Updates applications without downtime.

Kubernetes architecture

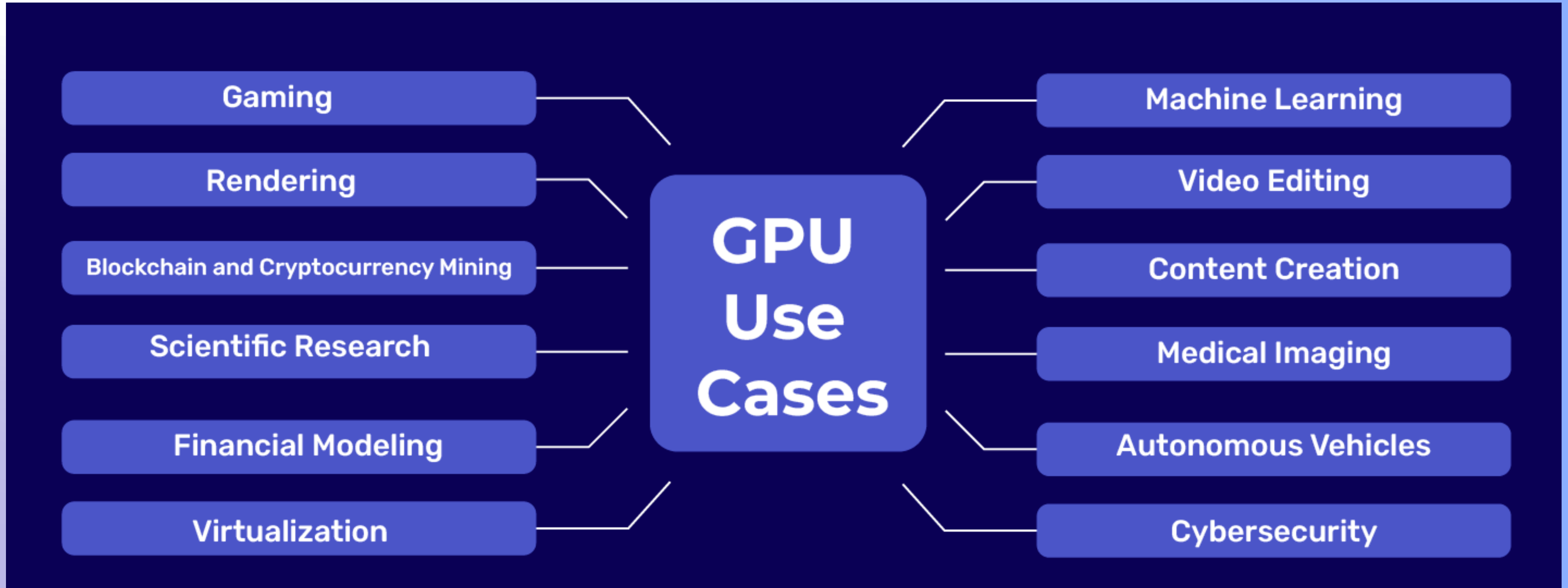


Flow of Working

Step-by-Step

- User sends command using kubectl/UI.
- Request reaches API Server.
- Scheduler selects suitable worker node.
- Controller Manager monitors execution.
- Pod is created on worker node.
- Docker/container runtime starts containers.
- kubelet checks pod health.
- kube-proxy manages networking.

MAJOR APPLICATIONS OF GPU



Parallel Computing for AI/ ML

- Parallel computing plays a very important role in Artificial Intelligence (AI) and Machine Learning (ML). AI/ML applications require huge amounts of data processing and complex mathematical computations, which can be time-consuming on a single processor.
- Parallel computing solves this problem by dividing tasks among multiple processors, GPUs, or computing nodes and executing them simultaneously.

In AI and ML, parallel computing is mainly used for:

- Training deep learning models
- Processing large datasets
- Image and speech recognition
- Natural Language Processing (NLP)
- Real-time predictions and analytics

GPUs and distributed systems help accelerate matrix operations, neural network training, and big data processing. Frameworks like TensorFlow, PyTorch, CUDA, and Hadoop support parallel execution for faster performance.

Parallel Computing for AI/ ML

Advantages

- Faster model training
- Efficient handling of big data
- Reduced computation time
- Better scalability and performance

Applications

- Self-driving cars
- Chatbots
- Recommendation systems
- Medical diagnosis
- Face recognition

THANK YOU!



SUBSCRIBE

